Credit Risk Monitor: solution overview

A risk dashboard for a retail loan book: **14,685 loans, 10,000 customers, 601.1M lent, 16.1% went bad**. Live app: credit-risk-monitor-romit.streamlit.app · Code: github.com/romitsoni/credit-risk-dashboard

1. What the dashboard shows, and how an analyst uses it

Tab	What you see	How a risk analyst uses it
Portfolio	Key numbers, money and share that went bad per risk group, compare by any field, two fields together (heatmap), by year, list of loans in a group with plain reasons, 5 insights	Morning check: where is the risk and the money? Filter (e.g. Mortgage + Very High) and drill down to the loans that need action
Account	Predicted risk from both models, top reasons with the customer's real values, suggested next step, 12 months of payments, loans at other banks, other loans with us	Before a call or review: understand *why* one loan is risky and what to do next
Ask the Portfolio	Question box + floating 'Ask' button on every tab, answers with a badge and the source table, ready-made reports	Quick answers without writing code, e.g. 'How much money is in each risk group?'
Anomaly Review	Flagged accounts, plain reason for each, payment chart, Confirm / Dismiss form, decision log	Work through the review list each day and record a decision on each account
Collections	Which collection actions bring money back, cost calculator, which late customers pay	Choose the cheapest action that works; see what data needs fixing
Model & Monitoring	Model scores, lift, accuracy check, top drivers, drift check (PSI), monitoring plan, fairness by gender	Monthly: is the model still accurate, stable and fair?

2. The two AI features

	AI 1: Ask the Portfolio (Google Gemini)	AI 2: spotting unusual accounts
How it works	AI call 1 picks one of 7 approved reports (it sees no data). Python works out the numbers. AI call 2 writes 1 to 3 sentences from one small summary table. Python then checks every number; if one does not match, a safe answer written by code is shown instead. No key or AI down: ready-made reports still work.	Compares each account with **its own past** (on time for 9 months then missed; days late up by 30+) and with **similar customers** (top 1% for credit used or days late at other banks, within the same loan type and income group), plus a **rare-mix** check (Isolation Forest, rarest 2%). 981 flagged; they go bad **1.5x as often** (23.3% vs 15.6%).
Trigger	The analyst asks a question	Every data refresh, on every account
False positive	It answers, but picks the wrong report (e.g. city tier instead of region). The source table always shows which report was used	A flag on a fine account, e.g. one late month because a salary came late
False negative	It says 'can't answer' to a question the data could answer. Reword it or use a ready-made report	A risky account not flagged, e.g. one that gets slowly worse without crossing any line
Human review	Check the badge and the source table before using a number	Read the reason, check the account, then Confirm or Dismiss with a note. Saved with name and time; never deleted

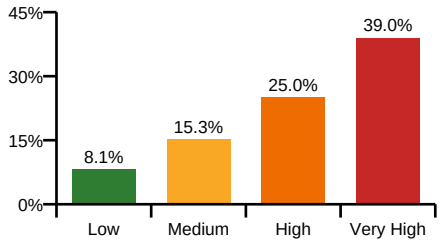
3. Model results and approach

Model (test set: customers never seen)	AUC	KS	Gini
Logistic regression (simple baseline)	0.663	0.256	0.325
XGBoost (used in the dashboard)	0.661	0.243	0.322

In plain words: AUC 0.66 means that if you take one bad loan and one good loan, the model gives the bad one the higher risk **66 times out of 100** (50 = guessing). KS and Gini measure the same separation on a 0 to 1 scale. The riskiest 20% of loans hold **36.7% of all bad loans** and **59% of the money at risk**. Predicted 15.7% vs real 16.1%, so the percentages can be trusted.

Approach: one row per loan from 5 joined tables; payments squashed into 12-month summaries; split by **customer** (75/25) so no customer is in both training and test; early stopping; SHAP for per-loan reasons. Collections, gender, marital status, IDs and dates are not inputs (unfair or 'cheating' data). 8 algorithms and 3 blends all scored 0.66 to 0.68, so the limit is the data, not the method. Risk groups: Low under 10%, Medium 10-20%, High 20-35%, Very High over 35% (average is 16%).

Share that really went bad, by risk group



Test customers only. Each group goes bad more than the one before.

4. Top 3 portfolio insights

Finding	What to do
1. Mortgages are the biggest worry. 24.7% go bad (vs 16.1% overall), and they hold 52% of the money lent and half of the expected loss.	Review how mortgages are approved first.
2. Income and age drive risk; region does not. Lowest income group 24.6% vs highest 9.7%; under-30s 21.6% vs over-60s 6.8%; regions only 15.0% to 16.9%. Low-income contract workers: 27.6%.	Set rules by the customer's profile, not by where they live.
3. Price does not follow risk. The interest rate alone predicts almost nothing (AUC 0.507): good and bad customers pay about the same rate.	Use the risk score for fairer, risk-based pricing.

5. Key assumptions, and what a bank's IT and data teams would need to provide

Assumptions	Needed for production
• The model scores **existing** loans with 12 months of payments; it is not for approving new customers • 'Went bad' was recorded after those 12 months • Other-bank data is a current snapshot; dates do not line up across tables, so they are not used • Money at risk = original loan amount; loss on default 45% (25% for mortgages and car loans) • Income is yearly; collection costs on that tab are examples, not real data	• **Data:** daily feeds with matching dates, the date of each default, 24+ months of history, other-bank data as of the approval date, current balances, collection results from the payment system • **Systems:** hosting inside the bank, single sign-on with roles, a database with an audit trail for decisions, scheduled score refresh, a secrets vault • **AI:** a business AI service that does not learn from bank data, logging of every question and answer, security review • **Oversight:** independent model validation, a named model owner, fair-lending review, approval for any retrain